Data-Level Parallelism in K-Means Clustering: Scalar C versus XMM and YMM SIMD Implementations

Project Proposal | Advanced Computer Architecture (CEPARCO), 1st Term AY 2026-2027 Group 2, Section S01

Daniel Jedrick C. Dabuit, Ethan Daniel B. Magbatoc, Jose Bryan L. Perez, Tara Ysabel P. Uy

0. Abstract

This project evaluates how much data-level parallelism improves the performance of k-means clustering. A sequential C implementation serves as the baseline and is compared against two SIMD implementations written in x86-64 assembly: one using XMM registers (128-bit, four single-precision values per instruction) and one using YMM registers (256-bit, AVX2, eight values per instruction). The parallelization targets the assignment step of Lloyd's algorithm, which computes the distance between every data point and every cluster centroid. The three versions are compared on execution time, speedup, and accuracy (agreement with the C result) across several numbers of points (N) and clusters (K). The goal is to show how SIMD register width affects k-means performance and where the speedup stops.

1. Description of the Proposed Project

K-means clustering partitions N points into K clusters. Starting from K initial centroids, it repeats two steps until the centroids stop moving or a maximum number of iterations is reached: (1) the assignment step labels each point with its nearest centroid, and (2) the update step moves each centroid to the mean of the points assigned to it. The distance used is the squared Euclidean distance, $d^2 = (x - cx)^2 + (y - cy)^2$. The square root is omitted because it does not change which centroid is nearest.

1.a Inputs

Input	Description
X[], Y[]	Single-precision (float32) coordinates of the N points, stored as two separate arrays (structure-of-arrays)
N	Number of data points

K	Number of clusters
MAX_ITER	Maximum number of iterations
TOL	Convergence threshold on centroid movement
SEED	Fixed random seed, so all versions receive identical data and identical initial centroids

The datasets are synthetic Gaussian blobs generated with a fixed seed. Planned sizes are N = 2^16, 2^20, and 2^24 with K = 4, 8, 16, and 32, plus boundary sizes (N = 7, 1003, 1005) to test the handling of leftover points. The initial centroids are the first K points of the array.

Input rules: X[] and Y[] must have the same length, $N \geq K \geq 1$, and every coordinate must be a valid finite number.

1.b Proposed Process

i. Processes to be parallelized

The **assignment step** is the main target. For each point, the squared distance to each centroid is computed and the smallest is kept. A point's result does not depend on any other point, so the step is data-parallel. With XMM registers, four points are processed per instruction; with YMM registers, eight. This step performs $N \times K$ distance computations per iteration and accounts for most of the work.

The **centroid accumulation** (per-cluster sums of x and y, and member counts) is a secondary target, to be vectorized only if time permits.

ii. Related works and how this project differs

Lloyd's algorithm is the standard formulation of k-means [1]. FAISS [2] is a library for similarity search and clustering of dense vectors, and its paper notes that GPUs are well suited to data-parallel tasks such as distance computation. NVIDIA's cuVS library [3] provides a GPU-accelerated k-means that follows the same two steps (assign each row to the closest centroid, then move each centroid to the average of its rows) and reports centroids, labels, inertia, and the iteration count. Its documentation notes that the expensive part is repeatedly comparing many rows against many centroids, which is the step targeted here.

These works rely on GPUs or tuned libraries. This project instead writes the SIMD kernels by hand in x86-64 assembly and compares 128-bit and 256-bit register widths against a scalar C baseline on the same CPU. The aim is not to outperform a production library but to show where the speedup originates and what limits it, such as memory bandwidth and the steps that remain sequential.

iii. Plan for parallelizing the process

The algorithm is transformed to expose SIMD parallelism as follows:

1. **Data layout.** Store X[] and Y[] as separate arrays so that one load fills a register with consecutive x (or y) values.
2. **Remove the square root.** Compare squared distances only.
3. **Broadcast.** For each centroid, copy cx and cy into every slot of a register.
4. **Distance.** Compute $dx = X - cx$, $dy = Y - cy$, and $d^2 = dx \cdot dx + dy \cdot dy$ for all points in the register at once, using separate multiply and add instructions (no fused multiply-add) so results match the C version exactly.
5. **Branch-free minimum.** Keep, per slot, the smallest d^2 so far and the index of the centroid that produced it, using a compare and a blend instead of a conditional jump. Ties go to the lower centroid index in all versions.
6. **Centroid loop.** Iterate over the K centroids sequentially, then store the labels of the whole group of points.
7. **Leftover points.** When N is not a multiple of 4 (XMM) or 8 (YMM), finish the remaining points with a scalar loop. For N = 1003, the YMM kernel performs 125 steps of 8 points and the scalar loop handles the last 3.

The centroid update is harder to vectorize because many points add into the same cluster total. One option is to process one cluster at a time: compare the labels against the cluster number, mask out other points, and sum the rest. This requires K passes and is only worthwhile for small K.

Three versions will be built: scalar C, XMM, and YMM. A CUDA (SIMT) version is an optional extension if time allows after the CUDA lecture.

iv. Processes that will not be parallelized

- Data generation and input validation
- Initial centroid selection
- The iteration loop and the convergence check (K comparisons per iteration)
- The division of the sums by the counts to obtain the new centroids (K divisions)
- Timing, accuracy checking, and reporting
- The centroid update, unless its vectorized version is completed

Because these parts remain sequential, the overall speedup is expected to be lower than that of the assignment step alone (Amdahl's law). Each stage is therefore timed separately.

1.c Outputs

- The final label of every point, the number of members in each cluster, and the final centroids
- Execution time per stage for each version: initial centroid selection, assignment with member counting, centroid recomputation, and full runtime
- Speedup of the XMM and YMM versions over C, summarized by the geometric mean across the tested sizes

- **Similarity score:** the percentage of points assigned the same label as in the C version (expected 100%), together with the largest centroid difference and the inertia (sum of squared distances to the centroids, as reported by cuVS [3]) of each version

Timing uses QueryPerformanceCounter, with one warm-up run followed by 30 timed runs averaged, in both Debug and Release configurations.

2. Proposed Target Implementation Platform

An x86-64 CPU supporting SSE and AVX2, running Windows. The driver, baseline, data generation, correctness check, and timing are written in C and built in Visual Studio. The SIMD kernels are written in x86-64 assembly (NASM) and linked with the C program. Instruction details are checked against the Intel Intrinsics Guide [4] and the Intel 64 and IA-32 Architectures Software Developer's Manual [5].

3. Mid-Project Milestones

Milestone #2 (Week 10): mid-project update demo

- Data generator and timing code completed
- Sequential C k-means completed and verified on a small hand-worked example
- XMM assignment kernel working and producing the same labels as the C version
- YMM kernel started, with preliminary C versus XMM timing results

Milestone #3 (Week 13): complete submission and demo

- YMM assignment kernel completed and verified
- Boundary sizes tested
- Full benchmarks across all N and K in Debug and Release
- Optional vectorized centroid update and optional CUDA version
- GitHub repository with source code, discussion of the parallel algorithms, and the sequential versus parallel execution time comparison
- Optional conference paper

4. References

1. S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Information Theory*, vol. 28, no. 2, pp. 129-137, 1982. https://www.stat.cmu.edu/~rnugent/PCMI2016/papers/LloydKMeans.pdf
2. J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, 2021. https://arxiv.org/abs/1702.08734
3. NVIDIA cuVS, "K-Means" clustering guide. https://docs.nvidia.com/cuvs/user-guide/api-guides/clustering-guide/k-means
4. Intel, *Intel Intrinsics Guide*. https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html

5. Intel, *Intel 64 and IA-32 Architectures Software Developer's Manual*. https://www.intel.com/content/www/us/en/content-details/825743/intel-64-and-ia-32-architectures-software-developer-s-manual-combined-volumes-1-2a-2b-2c-2d-3a-3b-3c-3d-and-4.html